MACHINE LEARNING

UNIT – III : Decision Trees and the Bayes Classifier

Easy digital notes with diagrams and worked examples for students (JNTU B.Tech, 23A31401T)

Topics in this unit

1. Decision trees for classification and impurity measures (entropy, Gini, error)
2. Properties of decision trees, overfitting and pruning
3. Regression based on decision trees
4. Bias-variance trade-off
5. Random forests for classification and regression
6. Bayes classifier: Bayes' rule, inference and optimality
7. Multi-class classification
8. Class conditional independence and the Naive Bayes classifier (NBC)

> * **Study tip:**
> Both halves of this unit use the same toy data - the 14-day Play-Tennis table. Do the ID3 and Naive Bayes calculations once by hand and every formula becomes easy.

Syllabus (as given)

Models Based on Decision Trees: Decision Trees for Classification, Impurity Measures, Properties, Regression Based on Decision Trees, Bias-Variance Trade-off, Random Forests for Classification and Regression. The Bayes Classifier: Introduction to the Bayes Classifier, Bayes' Rule and Inference, The Bayes Classifier and its Optimality, Multi-Class Classification, Class Conditional Independence and Naive Bayes Classifier (NBC).

Part I : Models Based on Decision Trees

3.1 Decision Trees for Classification

A **decision tree** is a flow-chart-like model that classifies an input by asking a **series of questions** about its features. Each question sends the input down one branch until it reaches a final answer.

- **Root node**: first test. **Internal node**: a test on one feature. **Branch**: outcome of the test. **Leaf node**: class label (the answer).
- **Prediction:** start at the root, follow the branch that matches the input, repeat, and output the label of the leaf reached.
- **Geometry:** every test like $x_j < t$ cuts the space with a line parallel to an axis, so the tree divides the feature space into **rectangular regions**, one class per region (Fig 3.1).

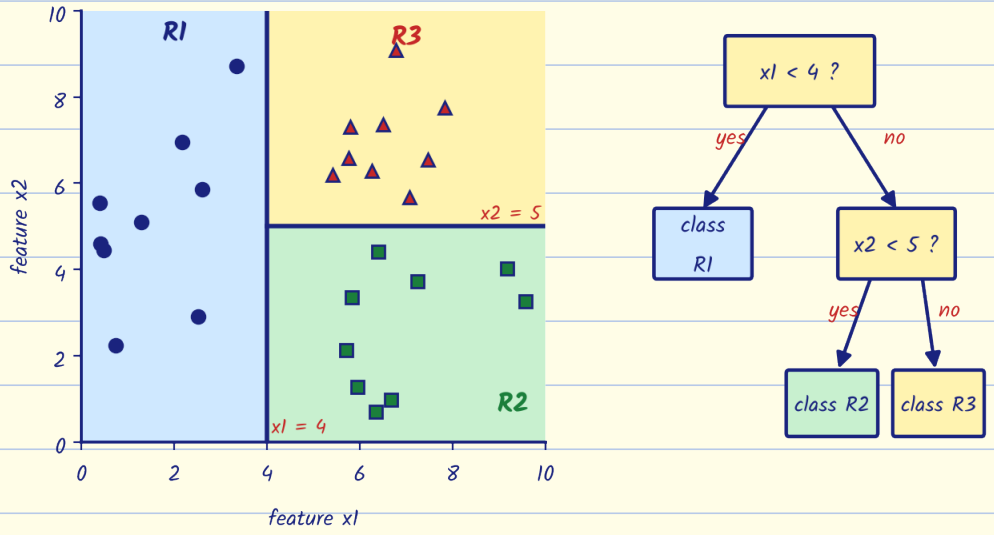


Fig 3.1 : A decision tree partitions the feature space into axis-parallel regions

Learning a tree: recursive partitioning (greedy, top-down)

1. Start with all training data at the root.
2. If the node is **pure** (one class) or a stopping rule fires, make it a **leaf** labelled with the **majority class**.
3. Otherwise try every feature (and threshold) and pick the split with the **largest impurity decrease** (best gain).
4. Split the data into child nodes and **repeat** the same steps on each child.

Algorithm	Split criterion	Features / output	Remarks
ID3 (Quinlan, 1986)	Information gain (entropy)	Categorical only; classification	Multi-way split; no pruning
C4.5	Gain ratio	Categorical + continuous; handles missing values	Post-pruning; successor of ID3
CART (Breiman, 1984)	Gini index (variance for regression)	Any features; classification + regression	Always **binary splits**; cost-complexity pruning

3.2 Impurity Measures

The best split is the one that makes the child nodes **as pure as possible**. An **impurity measure** $I(t)$ is 0 for a pure node and largest when classes are mixed equally. Let p_k = fraction of class k in node t.

Measure	Formula	Pure node	Max (2 classes)
Entropy	$H(t) = -\sum p_k \log_2 p_k$	0	1 ($\log_2 K$ for K classes)
Gini index	$Gini(t) = 1 - \sum p_k^2$	0	0.5 ($1 - 1/K$ for K classes)
Misclassification error	$E(t) = 1 - \max_k p_k$	0	0.5

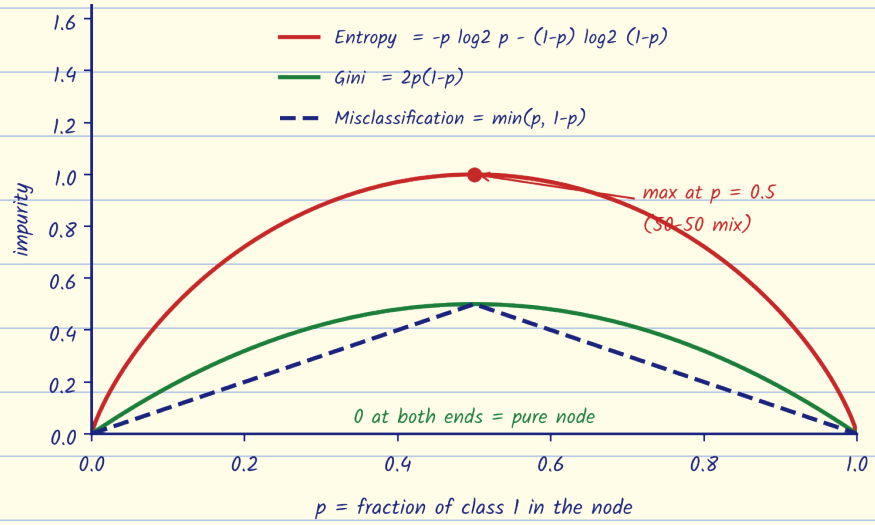


Fig 3.2 : Entropy, Gini and misclassification error for two classes

Node [Yes, No]	Entropy	Gini	Error
[7, 0]	0.000	0.000	0.000
[6, 1]	0.592	0.245	0.143
[9, 5]	0.940	0.459	0.357
[4, 3]	0.985	0.490	0.429
[5, 5]	1.000	0.500	0.500

- **Information gain** of splitting node S on feature A: $Gain(S, A) = H(S) - \Sigma_v (|S_v| / |S|) \cdot H(S_v)$ i.e. parent impurity minus **weighted** child impurity. Using Gini gives the **Gini decrease**.
- **Gain ratio** (C4.5): GainRatio = Gain / SplitInfo, $SplitInfo = - \Sigma_v (|S_v|/|S|) \log_2 (|S_v|/|S|)$. It punishes features with **many values** (e.g. an ID column has huge gain but is useless).
- **Misclassification error** is not smooth, so it is a poor choice for **growing** a tree, but it is common for **pruning**. Entropy and Gini are strictly concave, so they reward any purer child.

> * **Study tip:**
> Entropy and Gini give almost the same trees in practice. Gini is cheaper (no logarithm), entropy is slightly more sensitive to class mix.

3.3 Worked Example : Building a Tree with ID3

Decide **PlayTennis** from 4 features. Data set: **14 days, 9 Yes and 5 No.**

Day	Outlook	Temp	Humid.	Wind	Play
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes

Day	Outlook	Temp	Humid.	Wind	Play
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Step 1 : entropy of the whole data set

$H(S) = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) =$ **0.940** bits. $Gini(S) = 1 - (9/14)^2 - (5/14)^2 =$ **0.459**. Error $= 5/14 = 0.357$.

Step 2 : gain of each feature at the root

Take **Outlook**: Sunny [2 Yes, 3 No], Overcast [4 Yes, 0 No], Rain [3 Yes, 2 No]. $H(Sunny) = H(Rain) = 0.971$, $H(Overcast) = 0$.

$Gain(S, Outlook) = 0.940 - (5/14)(0.971) - (4/14)(0) - (5/14)(0.971) = 0.940 - 0.694 =$ **0.247**

Feature	Gain (entropy)	Gini decrease	SplitInfo	Gain ratio
Outlook	0.247	0.116	1.577	0.156
Temp	0.029	0.019	1.557	0.019
Humidity	0.152	0.092	1.000	0.152
Wind	0.048	0.031	0.985	0.049

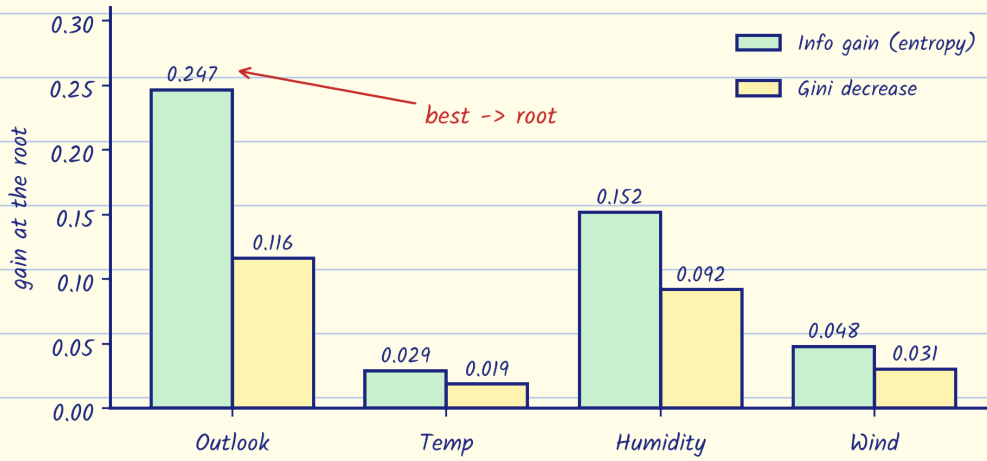


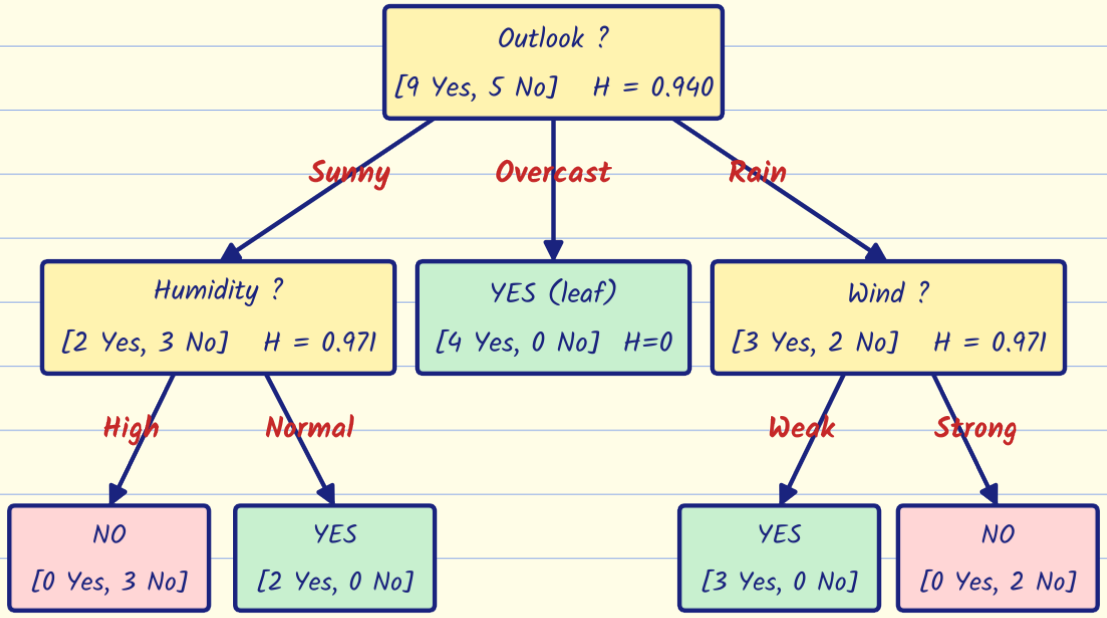
Fig 3.3 : Information gain and Gini decrease at the root: Outlook wins

Outlook has the highest gain (0.247) so it becomes the **root**. Overcast is already pure, so it becomes a leaf (Yes).

Step 3 : repeat on the impure branches

- **Sunny** [2 Yes, 3 No]: gains Temp = 0.571, Humidity = **0.971**, Wind = 0.020. Humidity separates perfectly (High -> No, Normal -> Yes).

- **Rain** [3 Yes, 2 No]: gains Temp = 0.020, Humidity = 0.020, Wind = **0.971**. Wind separates perfectly (Weak -> Yes, Strong -> No).
- All leaves are pure, so we **stop**. Temperature is never used - the tree has **selected features** automatically.



root = top node | internal node = a test | leaf = final answer (class)

Fig 3.4 : Final ID3 tree for Play-Tennis (5 pure leaves)

- **Rules read from the tree:** IF Overcast THEN Yes; IF Sunny AND Humidity = High THEN No; IF Sunny AND Humidity = Normal THEN Yes; IF Rain AND Wind = Weak THEN Yes; IF Rain AND Wind = Strong THEN No.
- **Predict** (Sunny, Cool, High, Strong): Outlook = Sunny -> Humidity = High -> **No**. Training error = 0.

3.4 Properties of Decision Trees

Property	Meaning
Non-parametric, interpretable	No fixed form assumed; the tree can be drawn and read as IF-THEN rules
Mixed data, little preparation	Handles numeric and categorical features; **no scaling needed**; invariant to monotonic transforms
Greedy search	Best split at each node only, so the tree is **not globally optimal** (finding the optimal tree is NP-complete)
Axis-parallel boundaries	Tilted boundaries need many small steps (staircase)
Automatic feature selection	Irrelevant features are simply never chosen for a split
High variance (unstable)	A small change in data can give a completely different tree
Overfitting	A fully grown tree reaches 100 % training accuracy by memorising noise
Bias to many-valued features	Information gain prefers features with many values - use **gain ratio**

Continuous features, missing values

- **Continuous feature:** sort the values, test thresholds at the **mid-points** between neighbouring values, and pick the best threshold t for the binary test $x < t$.
- **Missing values:** send the example down all branches with fractional weights (C4.5), or use surrogate splits (CART), or impute first.

Overfitting and pruning

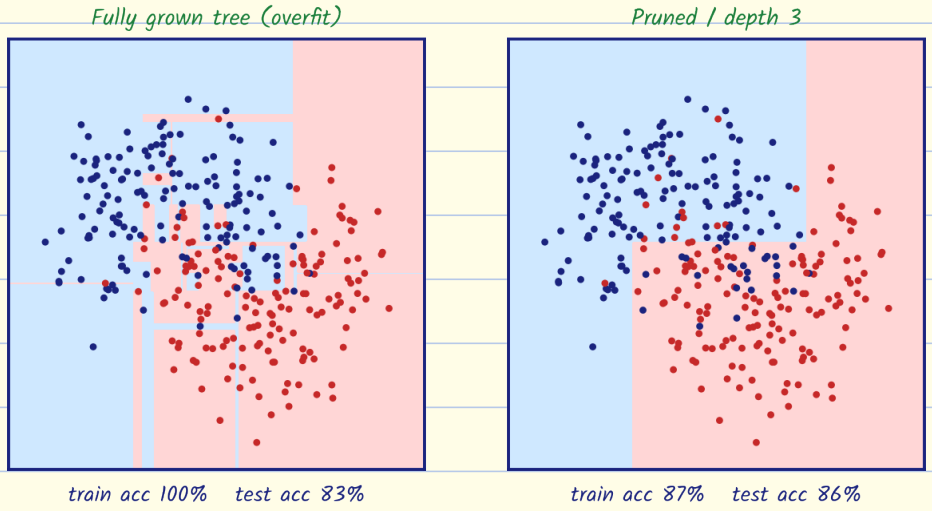


Fig 3.5 : Fully grown tree overfits; a shallower tree generalises better

- **Pre-pruning (early stopping):** stop growing when depth reaches a limit, a node has fewer than a minimum number of samples, or the best gain is below a threshold.
- **Post-pruning:** grow the full tree, then cut back subtrees that do not help on validation data. **Reduced-error pruning** replaces a subtree by a leaf if validation error does not rise.
- **Cost-complexity pruning (CART):** minimise $R_\alpha(T) = R(T) + \alpha\,|T|$, where $R(T)$ = training error and $|T|$ = number of leaves. Larger α gives a smaller tree; α is chosen by cross-validation.
- **Hyper-parameters:** max_depth, min_samples_leaf, min_impurity_decrease, ccp_alpha.

3.5 Regression Based on Decision Trees

When the target is a **real number**, the tree is a **regression tree** (CART). The leaf prediction is the **mean** of the training targets in that leaf, so the fitted function is a **step function**.

- **Impurity:** node variance, i.e. the **sum of squared errors** $SSE(t) = \Sigma\,(y_i - \bar{y}_t)^2$ with $\bar{y}_t$ = mean of y in t.
- **Best split** (feature j, threshold s) minimises SSE(left) + SSE(right), i.e. gives the largest **variance reduction**: $\Delta SSE = SSE(parent) - SSE(left) - SSE(right)$.
- **Prediction** for x falling in leaf R_m: $\hat{y}(x) = \bar{y}_{R_m} = (1/N_m)\,\Sigma_{x_i \in R_m}\, y_i$.
- **Stopping and pruning** are the same as for classification trees (depth, minimum samples, α-pruning with squared error).

Worked example : 8 points, one feature

x	1	2	3	4	5	6	7	8
y	4	5	4	6	15	16	14	15

SSE(parent) = 214.88 (mean = 9.875). Try thresholds: $x \le 1.5$: 175.43, $x \le 2.5$: 137.83, $x \le 3.5$: 67.47, $x \le 4.5$: 4.75, $x \le 5.5$: 88.80, $x \le 6.5$: 157.83, $x \le 7.5$: 184.86. Best is **$x \le 4.5$** with SSE = 2.75 + 2.00 = 4.75, so variance reduction = 214.88 - 4.75 = **210.12**.

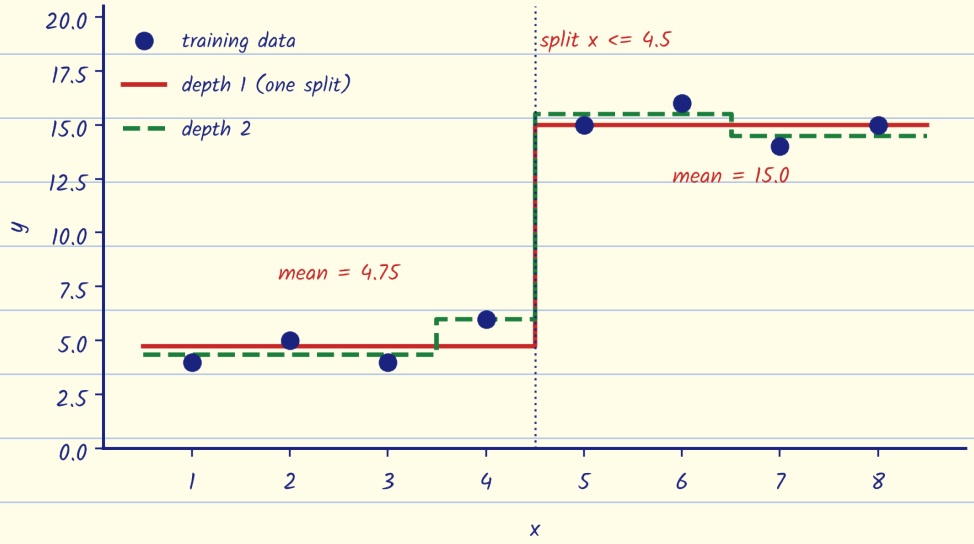


Fig 3.6 : Regression tree = step function; leaves predict means (4.75 and 15.0)

- The tree predicts **4.75** for $x \le 4.5$ and **15.0** for $x > 4.5$. Depth 2 adds more steps and follows the data more closely (lower bias, higher variance).
- **Limitations:** predictions are piecewise constant, cannot extrapolate beyond the training range, and the tree is unstable.

3.6 Bias-Variance Trade-off

Suppose $y = f(x) + \varepsilon$, with noise variance σ^2. A learner trained on a random training set gives a prediction $\hat{f}(x)$. Over many training sets, the expected squared error at x splits into three parts:

Bias-variance decomposition
$E[(y - \hat{f}(x))^2] =$ **Bias²** + **Variance** + σ^2
Bias = $E[\hat{f}(x)] - f(x)$ (systematic error: how far the average model is from the truth)
Variance = $E[(\hat{f}(x) - E[\hat{f}(x)])^2]$ (how much the model changes with the training set)
σ^2 = irreducible noise (no model can remove it).

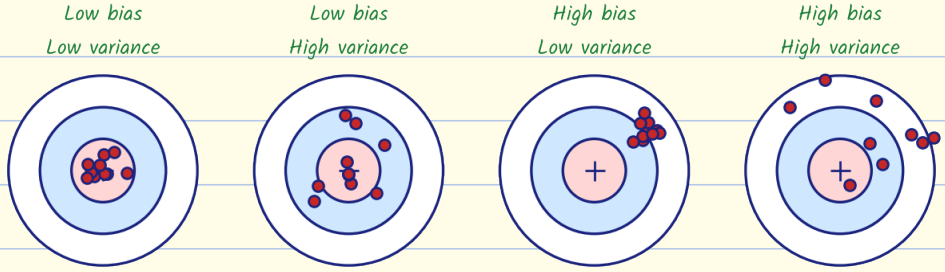


Fig 3.7 : Dartboard view: bias = distance of the cluster from the centre, variance = spread

	High bias (underfit)	High variance (overfit)
Model	Too simple (shallow tree, stump)	Too complex (deep tree)
Training error	High	Very low
Test error	High, close to training error	High; big gap from training error
Fix	Deeper tree, more features, more complex model	Prune, more data, **bagging / random forest**, regularisation

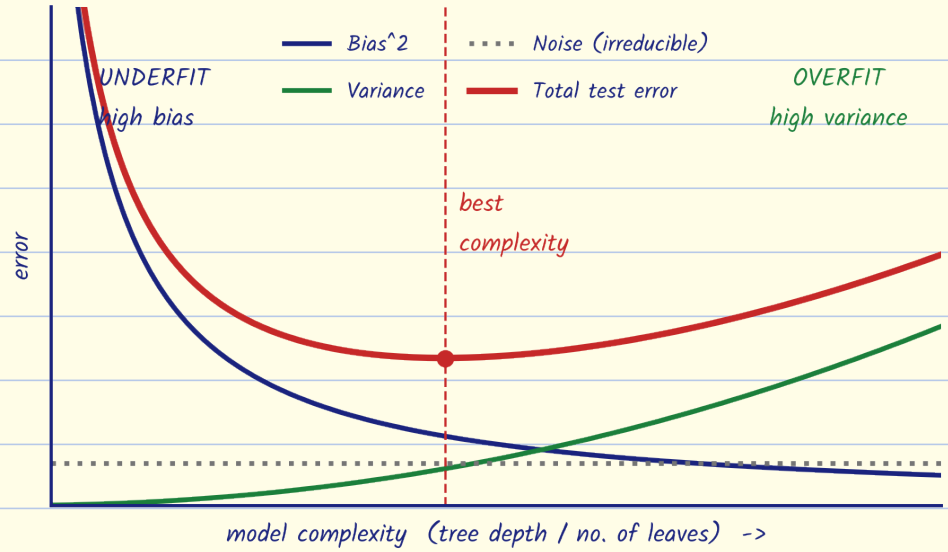


Fig 3.8 : Error against complexity: total error is lowest at the best depth

- **Trade-off:** a deeper tree **lowers bias** but **raises variance**; test error is U-shaped. The best depth is chosen on a validation set or by cross-validation.
- **Single decision tree** = low bias, **high variance**. Averaging many trees (random forest) cuts variance without raising bias much - this is the idea of the next section.

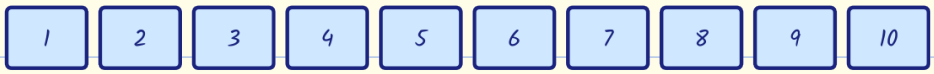
3.7 Random Forests for Classification and Regression

A **random forest** (Breiman, 2001) is an **ensemble** of many decision trees, each trained on a slightly different data set and a random choice of features. The trees' outputs are combined, which **reduces variance**.

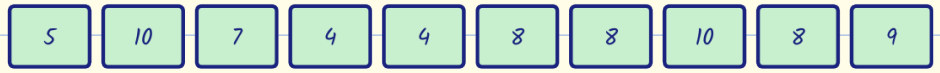
Step 1 : Bagging (bootstrap aggregating)

- Draw B **bootstrap samples**: each has N rows picked from the N training rows **with replacement**. A row is missed with probability $(1 - 1/N)^N \approx e^{-1} = 0.368$, so each sample holds about **63.2 %** of the distinct rows.
- Grow one tree on each sample - **deep and unpruned** (low bias, high variance).

Original data (10 rows):



One bootstrap sample (draw 10 WITH replacement):



Left out (out-of-bag, OOB) rows: 1, 2, 3, 6 -> used as a free test set

Fig 3.9 : A bootstrap sample and its out-of-bag (OOB) rows

Step 2 : Random feature subsets

- At **every split**, consider only m randomly chosen features out of d, and take the best of those. Typical: $m = \sqrt{d}$ for classification, $m = d/3$ for regression.
- **Why?** Without it, one strong feature sits at the top of every tree and the trees look alike (high correlation). Random subsets **de-correlate** the trees.

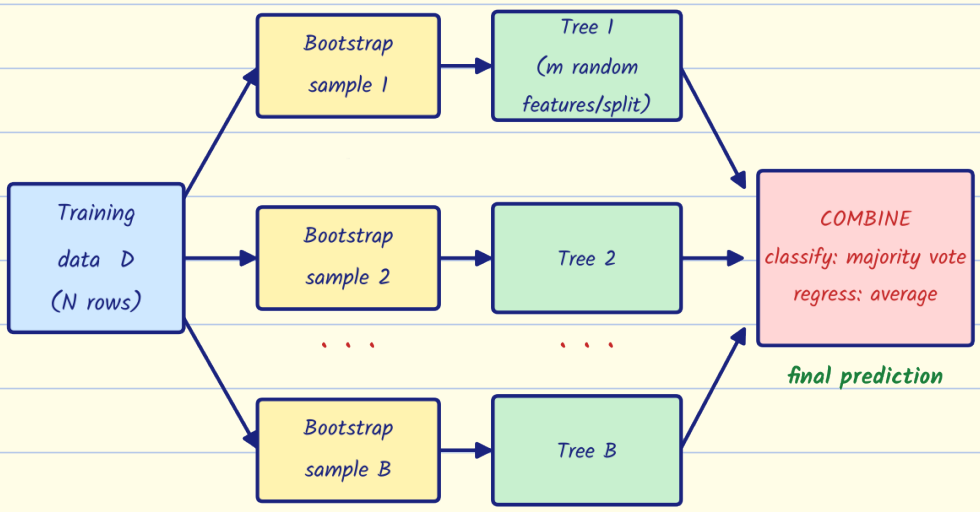


Fig 3.10 : Random forest: bootstrap, random-feature trees, combine

Step 3 : Combine

	Classification	Regression
Each tree outputs	A class label	A number
Forest output	Majority vote: $\hat{y} = \text{mode}\{ h_1(x), \ldots, h_B(x) \}$	Average: $\hat{y} = (1/B) \Sigma h_b(x)$
Tree split rule	Gini / entropy	Variance reduction (SSE)
Default m	$\sqrt{d}$	$d/3$

- **Why it works:** if the B trees have variance σ^2 and pairwise correlation ρ, the average has variance $\rho\sigma^2 + (1 - \rho)\sigma^2/B$. More trees kill the second term; random features lower ρ. Bias stays about the same as one deep tree.
- **Out-of-bag (OOB) error:** each tree never saw about 37 % of the rows. Predict each row using only the trees that left it out; the error over all rows is an **honest test-error estimate** - no separate validation set needed.

- **Feature importance:** (a) **mean decrease in impurity** - total impurity drop caused by splits on the feature, averaged over trees; (b) **permutation importance** - shuffle one feature and see how much OOB accuracy falls.

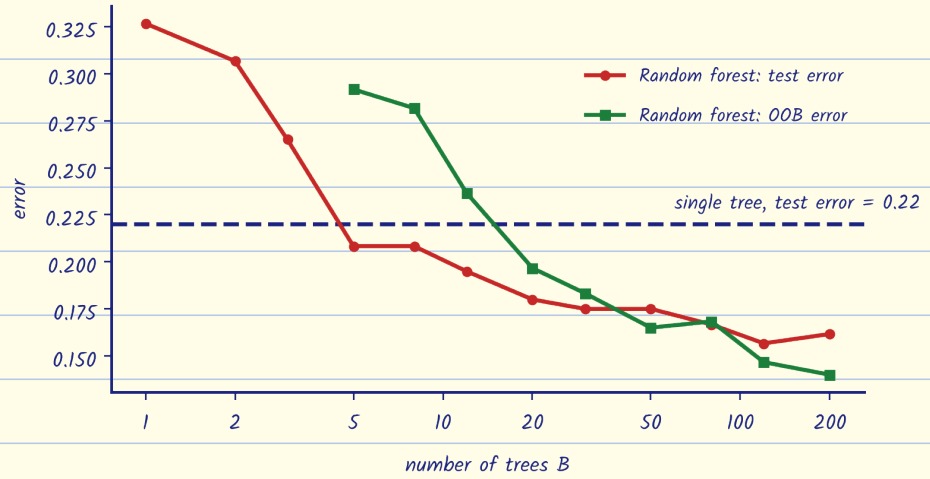


Fig 3.11 : Test and OOB error fall as trees are added and beat a single tree

- **More trees never cause overfitting** (error flattens); the main hyper-parameters are B, m, max_depth and min_samples_leaf.
- **Pros:** high accuracy, robust to noise and outliers, works on large data, gives importance. **Cons:** less interpretable than one tree, slower, large memory.

> * **Remember:**
> Tree: low bias, high variance. Random forest = many trees + bootstrap + random features + average/vote -> low bias, LOW variance.

Part II : The Bayes Classifier

3.8 Introduction to the Bayes Classifier

The Bayes classifier is a **probabilistic** classifier. For an input x it computes the **posterior probability** $P(C_k \mid x)$ of every class and picks the most probable class. Everything rests on a few probability rules.

Term	Meaning
Prior P(C)	Belief in class C before seeing any features (class proportion in the data)
Likelihood P(x | C)	How probable the features x are when the class is C
Evidence P(x)	$P(x) = \Sigma_k P(x \mid C_k) P(C_k)$ (total probability); same for every class
Posterior P(C | x)	Probability of class C after seeing x - this is what we want
Product rule	$P(A, B) = P(A \mid B) P(B) = P(B \mid A) P(A)$

3.9 Bayes' Rule and Inference

Bayes' rule
$P(C_k \mid x) = P(x \mid C_k) \cdot P(C_k) \,/\, P(x)$ posterior = likelihood × prior / evidence

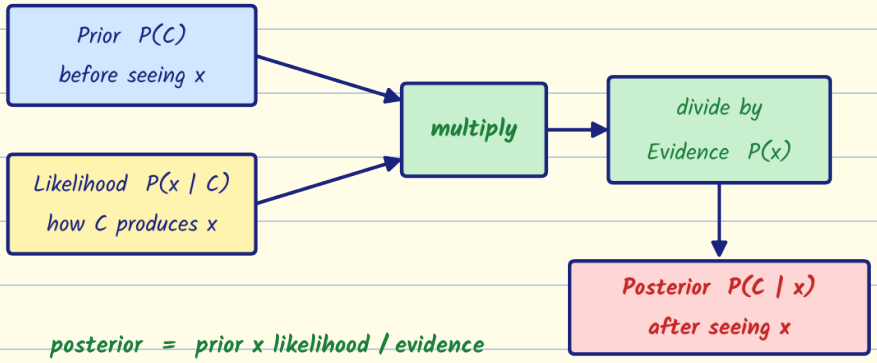


Fig 3.12 : Bayes' rule: prior and likelihood combine into the posterior

- **Inference** = updating belief: start with the prior, observe x, get the posterior (which becomes the new prior if more evidence arrives).
- **MAP (maximum a posteriori)** decision: $\hat{C} = \text{argmax}_k P(C_k \mid x) = \text{argmax}_k P(x \mid C_k) P(C_k)$. The evidence $P(x)$ is the same for all k, so it is **dropped** when only the winner is needed.
- **ML (maximum likelihood)** decision: $\text{argmax}_k P(x \mid C_k)$. It equals MAP when all priors are equal.

Worked example : medical test

Disease prevalence $P(D) = 0.01$. Test: $P(+ \mid D) = 0.95$, false-positive rate $P(+ \mid \text{no } D) = 0.05$. A patient tests positive. What is $P(D \mid +)$?

$P(+) = 0.95 \cdot 0.01 + 0.05 \cdot 0.99 = 0.0095 + 0.0495 = 0.0590$. $P(D \mid +) = 0.0095 \,/\, 0.0590 =$ **0.161**

* **Remember:**
Only about 16 % even after a positive test, because the disease is rare (low prior). Priors matter - do not ignore them.

3.10 The Bayes Classifier and its Optimality

The **Bayes classifier** assigns x to the class with the **largest posterior**:

* **Key idea:**
$f^*(x) = \text{argmax}_k P(C_k \mid x) = \text{argmax}_k P(C_k)\, p(x \mid C_k)$ (for continuous x use density $p(x \mid C_k)$)

- **Decision boundary:** the points where two posteriors are equal, e.g. $P(C_1)\, p(x|C_1) = P(C_2)\, p(x|C_2)$. In Fig 3.13 (equal priors, equal variance) it is the midpoint $x^* = 0$.
- **Error of any classifier h** at x: $P(\text{error} \mid x) = 1 - P(h(x) \mid x)$. This is smallest when $h(x)$ is the class with the **maximum** posterior. Hence for the Bayes classifier $P(\text{error} \mid x) =$ **$1 - \max_k P(C_k \mid x)$**.

Optimality theorem
Under 0-1 loss, no classifier (tree, neural network, anything) can have a smaller expected error than the Bayes classifier: $P(\text{error of } h) \geq P(\text{error of } f^*) = E_x[\, 1 - \max_k P(C_k \mid x)\,]$ = **Bayes error rate**.

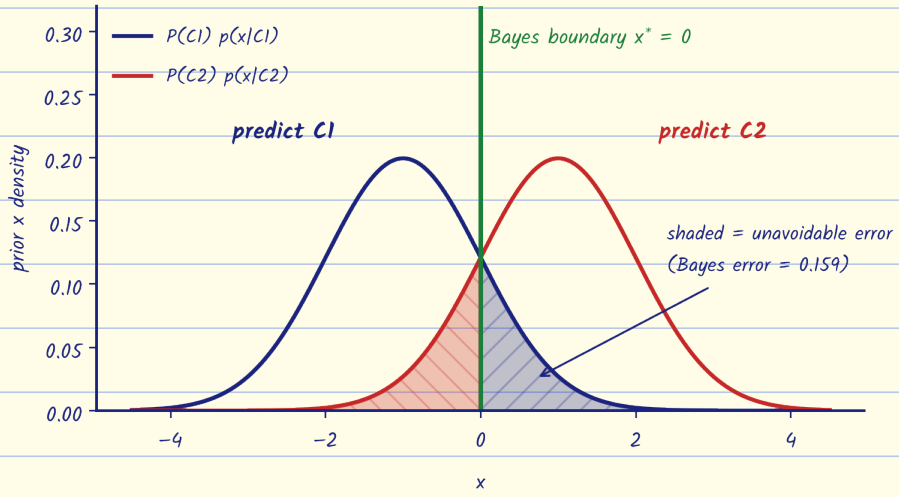


Fig 3.13 : Two classes with equal priors: Bayes boundary and the unavoidable (Bayes) error

- **Proof idea:** for fixed x, $P(\text{correct} \mid x) = P(h(x) \mid x) \le \max_k P(C_k \mid x)$, with equality when h picks the arg-max. Averaging over x keeps the inequality.
- **Bayes error rate** = the **irreducible** error caused by class overlap. Example in Fig 3.13: N(-1,1) vs N(+1,1), equal priors: error = $\Phi(-1)$ = **0.159** (the shaded area).
- **General loss:** with loss $\lambda(a_i \mid C_k)$ choose the action with the least **expected risk** $R(a_i \mid x) = \Sigma_k \lambda(a_i \mid C_k) P(C_k \mid x)$; 0-1 loss gives the MAP rule. (Useful when a missed cancer costs more than a false alarm.)
- **Practical catch:** the true $P(x \mid C_k)$ is unknown, so the Bayes classifier is a **theoretical benchmark**. Real methods estimate the probabilities - Naive Bayes does it with an independence assumption.

3.11 Multi-Class Classification

With K classes $C_1 \dots C_K$ the Bayes rule needs **no change**: compute K scores $g_k(x) = P(C_k)\, p(x \mid C_k)$ and choose the largest. The posteriors are the scores **normalised to sum to 1**: $P(C_k \mid x) = g_k(x) / \Sigma_j g_j(x)$.

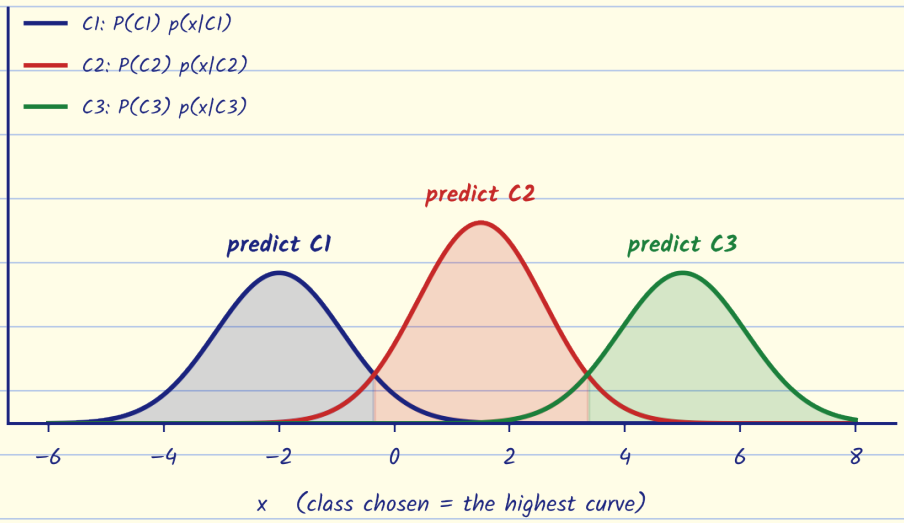


Fig 3.14 : Three classes: the class whose weighted density is highest wins; the line-up gives the decision regions

- **Decision regions** $R_k = \{x : g_k(x) \ge g_j(x) \text{ for all } j\}$; boundaries lie where the top two scores tie.
- **Error** = $1 - E[\max_k P(C_k \mid x)]$. It is easier to compute through the **correct** probability than by summing $K(K-1)$ misclassification terms.

- **Tree vs Bayes:** decision trees and random forests also handle K classes directly (leaf = majority class; Gini/entropy use all K proportions). Binary methods such as SVM need one-vs-rest or one-vs-one; the Bayes classifier does not.
- **Evaluation:** use a K × K confusion matrix; accuracy = diagonal sum / total.

3.12 Class Conditional Independence and the Naive Bayes Classifier

For d features, estimating the full likelihood $P(x_1, ..., x_d \mid C)$ needs a table of size $2^d - 1$ per class (binary features) - impossible for large d. **Naive Bayes** makes one simple assumption:

Class conditional independence
Given the class, the features are independent: $P(x_1, x_2, ..., x_d \mid C) = \Pi_{j=1..d} P(x_j \mid C)$

General (features depend on each other) Naive Bayes (conditionally independent)

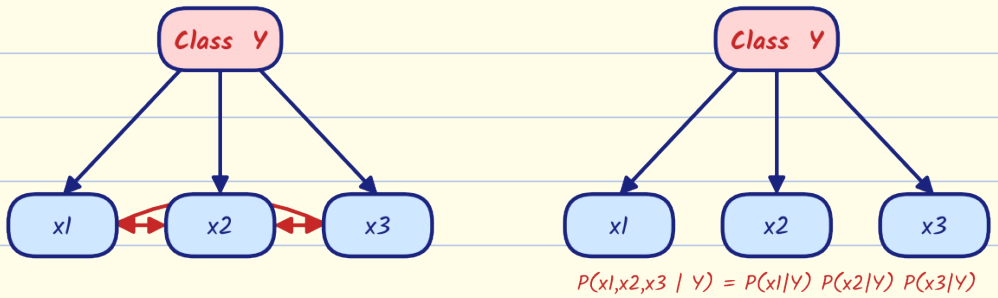


Fig 3.15 : Naive Bayes: the class alone explains each feature; no arrows between features

Naive Bayes classifier (NBC)
$\hat{C} = \text{argmax}_k \, P(C_k) \cdot \Pi_{j=1..d} P(x_j \mid C_k)$

- **Training** = just **counting**: $P(C_k) = N_k/N$ and $P(x_j = v \mid C_k) = \text{count}(x_j = v, C_k) / N_k$. One pass over the data; parameters grow as d per class, not 2^d.
- **Why "naive"?** Features are rarely truly independent (humidity and temperature), yet the **arg-max is often still right** because only the ranking of classes matters, not exact probabilities.
- **Use** small products as **log-sums**: $\log P(C_k) + \Sigma \log P(x_j \mid C_k)$ to avoid numerical underflow.

Worked example 1 : Play-Tennis, x = (Sunny, Cool, High, Strong)

Quantity	Class = Yes (9 rows)	Class = No (5 rows)
Prior P(C)	9/14	5/14
P(Sunny | C)	2/9	3/5
P(Cool | C)	3/9	1/5
P(High | C)	3/9	4/5
P(Strong | C)	3/9	3/5
Score = product	0.00529	0.02057
Posterior (normalised)	0.205	**0.795**

Yes: (9/14)(2/9)(3/9)(3/9)(3/9) = 0.00529. No: (5/14)(3/5)(1/5)(4/5)(3/5) = 0.02057. The larger is **No**, so the prediction is **do not play** (posterior 0.80); same answer as the decision tree.

The zero-frequency problem and Laplace smoothing

New day x = (**Overcast**, Cool, High, Strong). In the data, Overcast never occurs with No, so P(Overcast | No) = 0/5 = 0 and the **whole product** for No becomes 0, however strong the other evidence is.

Laplace (add-one) smoothing: $P(x_j = v \mid C) = (\text{count}(v, C) + \alpha) / (N_C + \alpha \cdot V_j)$, $\alpha = 1$, V_j = number of values of feature j (Outlook 3, Temp 3, Humidity 2, Wind 2). Prior is left unsmoothed here.

Factor	Without smoothing	With Laplace (α = 1)
Yes: P(Over|Yes), P(Cool|Yes), P(High|Yes), P(Strong|Yes)	4/9, 3/9, 3/9, 3/9	5/12, 1/3, 4/11, 4/11
No: P(Over|No), P(Cool|No), P(High|No), P(Strong|No)	**0/5**, 1/5, 4/5, 3/5	1/8, 1/4, 5/7, 4/7
Score Yes	0.0106	0.0118
Score No	**0**	0.0046
Posterior Yes / No	1.00 / 0.00	0.722 / 0.278

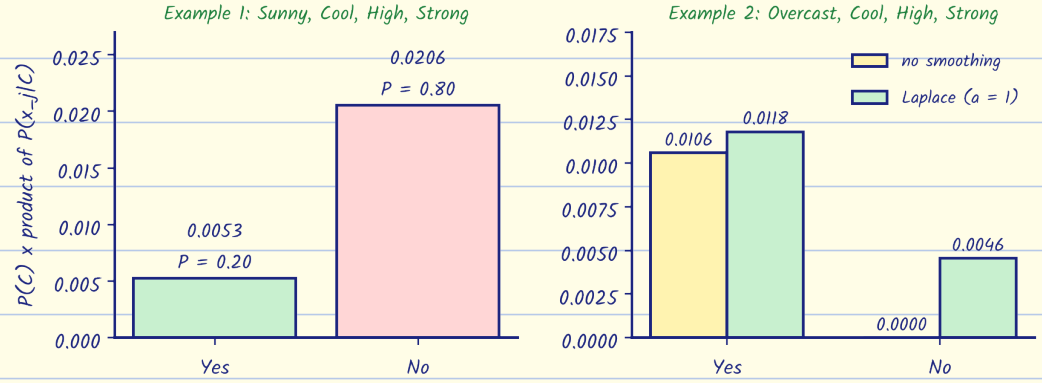


Fig 3.16 : NBC scores: Example 1 (left) and the zero-frequency case with and without smoothing (right)

- Without smoothing the model says Yes with **100 % certainty** just because one count is 0 - over-confident. With smoothing the zero disappears and the posterior is a sensible **0.72** for Yes; the decision (Yes) is the same.
- Larger α pulls every estimate toward uniform; $\alpha \to 0$ gives the raw frequencies. α is a hyper-parameter.

Naive Bayes for continuous features : Gaussian NB

For a numeric feature, assume $P(x_j \mid C_k)$ is a **Gaussian** whose mean μ_{jk} and variance σ_{jk}^2 are estimated from the class-k training values: $p(x_j \mid C_k) = (1 / \sqrt{(2\pi\ \sigma^2)}) \cdot \exp(- (x_j - \mu)^2 / (2\sigma^2))$.

Example: temperature of Yes days = 70, 72, 74 ($\mu = 72$, $\sigma^2 = 2.67$); No days = 80, 85, 90 ($\mu = 85$, $\sigma^2 = 16.67$). For x = 75: p(75 | Yes) = 0.0452, p(75 | No) = 0.0049. With equal priors the ratio is 9.3 : 1, so predict **Yes**.

Feature type	Model for $P(x_j \mid C)$	Variant
Categorical / binary	Counts with Laplace smoothing	Multinomial / Bernoulli NB (text, spam)
Continuous	Gaussian $N(\mu, \sigma^2)$ per class	Gaussian NB

Advantages of NBC	Limitations of NBC
Very fast training and prediction; one pass of counting	Independence assumption is unrealistic; correlated features are double-counted
Works with small data and many features (text, spam filtering)	Probability outputs are poorly calibrated (over-confident); use only the ranking
Handles missing features (skip the factor); easy to update	Zero-frequency needs smoothing; cannot learn feature interactions

	Decision tree / forest	Naive Bayes
Type	Discriminative: learns the boundary directly	Generative: models $P(x \mid C)$ and $P(C)$, then applies Bayes' rule
Feature interactions	Captured by successive splits	Ignored (independence)
Data needed	Moderate to large	Small is enough
Interpretability	Rules (tree); low for forest	Per-feature probabilities

Quick Revision – Unit III

- **Decision tree:** root, internal nodes (tests), branches, leaves (class). Built greedily by recursive partitioning; axis-parallel regions.
- **Impurity:** Entropy $= -\Sigma\, p \log_2 p$; Gini $= 1 - \Sigma\, p^2$; Error $= 1 - \max p$. All 0 for a pure node, maximum at an even mix.
- **Gain** = H(parent) - weighted H(children); **gain ratio** = Gain / SplitInfo. ID3 (entropy), C4.5 (gain ratio), CART (Gini, binary).
- **Play-Tennis:** $H(S) = 0.940$; Gain(Outlook) = 0.247 (root), Humidity = 0.152, Wind = 0.048, Temp = 0.029.
- **Properties:** interpretable, no scaling, greedy, unstable, overfits. **Pruning:** pre (depth, min samples), post (reduced-error, cost-complexity $R(T) + \alpha|T|$).
- **Regression tree:** leaf = mean; split minimises SSE (variance reduction); piecewise-constant, no extrapolation.
- **Bias-variance:** $E[\text{error}] = \text{Bias}^2 + \text{Variance} + \sigma^2$. Complexity up -> bias down, variance up; test error is U-shaped.
- **Random forest:** bagging (bootstrap, 63.2 % rows) + random m features per split ($\sqrt{d}$ / $d/3$); vote or average; OOB error; feature importance; $\text{Var} = \rho\sigma^2 + (1 - \rho)\sigma^2/B$.
- **Bayes rule:** posterior = likelihood × prior / evidence. **MAP:** argmax $P(x|C)P(C)$.
- **Bayes classifier:** $f^*(x) = \text{argmax}\, P(C_k \mid x)$; **optimal** under 0-1 loss; Bayes error $= E[1 - \max P(C_k \mid x)]$ (irreducible).

- **Multi-class:** same rule with K scores, normalised to give posteriors.
- **Naive Bayes:** class-conditional independence, $P(x|C) = \prod P(x_j|C)$; train by counting; **Laplace** (count + 1)/(N_C + V) fixes zero frequency; **Gaussian NB** for numeric features.

Likely exam questions

1. What is a decision tree? Explain how a decision tree is constructed for classification. Compare ID3, C4.5 and CART.
2. Define entropy, Gini index and misclassification error. Compute the information gain of Outlook for the Play-Tennis data.
3. Construct a decision tree for a given small data set using information gain (show every gain calculation).
4. List the properties of decision trees. What is overfitting? Explain pre-pruning and post-pruning.
5. Explain regression trees. How is the best split chosen? Work out a split for a small numeric data set.
6. Explain the bias-variance trade-off and the decomposition of expected error with a diagram.
7. Explain random forests: bagging, random feature selection, OOB error and feature importance. Why does a forest have lower variance than a tree?
8. State Bayes' rule. Show that the Bayes classifier is optimal and define the Bayes error rate. Explain Naive Bayes with Laplace smoothing and classify a given sample.